

CPT202 2024/2025 Semester 2
Final Report for Software Engineering Group
Project

An online meeting booking system

May 19, 2025

<http://121.43.36.102/>

Group number: <17>

Student Name: Nan.Lin

Student ID: 2252173

Contents

Introduction	3
Software Development Process	4
Software Design	5
Change Management	9
Legal, social, ethical, and professional issues	9
Conclusion	10
References (if needed. Not count into the max page limit)	11
Appendix	12

Introduction

This report documents my personal experience in the development of the Online Meeting Room Booking System at Xi'an Jiaotong-Liverpool University. The system is designed to address common issues in traditional meeting room booking processes, such as manual errors, low efficiency, and delayed administrative responses. It offers a digital and intelligent solution aimed at improving resource utilization, enhancing user experience, and significantly boosting the operational efficiency of meeting room management. The final delivered system supports role-based access control for various user types, including students, teachers, MRC coordinators, maintenance staff, and super administrators. Key functionalities include online room booking and approval workflows, booking record management, real-time monitoring of room status, fault reporting, and data-driven usage analysis with visualized statistics. Through centralized management and intelligent scheduling, the system effectively reduces resource waste, increases the transparency and efficiency of the booking process, and provides robust data support for future operational analysis and decision-making (Georgeta *et al.*, 2018).

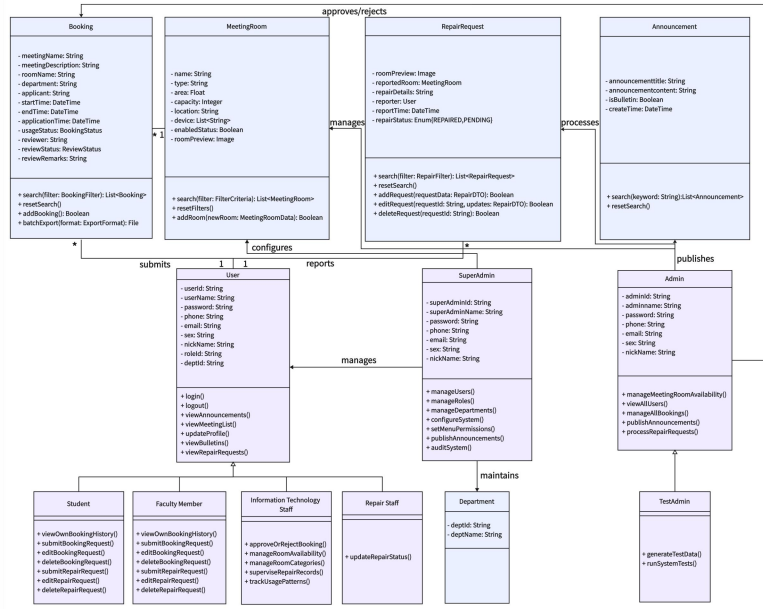


Figure 1-Appendix 1: System module design class diagram

In the early stage of the project, I collaborated with the project lead to define the development roadmap and management standards, including the specification and naming conventions for Product Backlog Items (PBIs). During the development phase, I was primarily responsible for the Meeting Room Management Module (Epic 03), which includes four essential features: adding, editing, deleting meeting rooms, and visualizing room usage statistics (PBI-011 to PBI-014). While the number of PBIs may seem limited, these functionalities serve as the structural core of the system, fundamentally supporting the booking workflow, administrative operations, and data transparency. To implement these features, I developed the front-end interfaces using Vue.js—a progressive JavaScript framework known for its reactive data binding and component-based architecture—enabling modular, maintainable, and responsive user experiences. I also contributed to front-end and back-end integration and participated in full-cycle testing. Collectively, these contributions improved the platform's usability and empowered administrators with greater efficiency in managing meeting room resources. In the later stages of the project, I also took charge of report writing, documentation compilation and

revision, and the production of the demonstration video, ensuring the deliverables were complete, well-documented, and ready for submission.

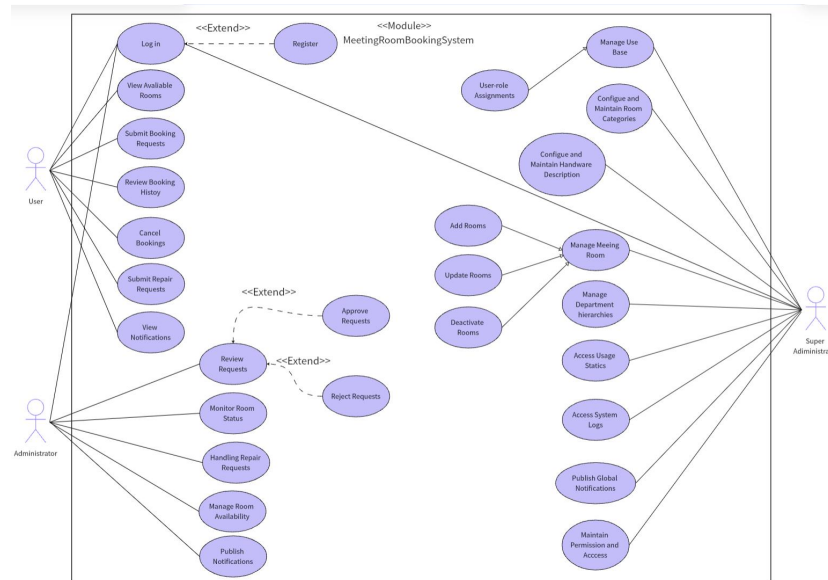


Figure 2-Appendix 2: Use case diagram of the system

This report follows the Scrum agile development framework and elaborates on the development process and my contributions across four key dimensions: Software Development Process, Software Design, Change Management, and Legal, Social, Ethical, and Professional Issues.

Software Development Process

Our team followed the Scrum methodology as outlined in [Appendix 3](#). Through regular Sprint planning, daily stand-ups, and review meetings, we improved development efficiency, adaptability, and transparency (Sutherland, 2014). I also contributed to maintaining the shared Sprint tracking document (see [Appendix 4](#)). In this project, I focused on the development of the Meeting Room Management module, including functionalities such as adding, editing, and deleting meeting rooms, as well as generating room usage statistics for administrators.

During Sprint 1, my key tasks were the development of the “Add Meeting Room” (PBI-011) and “Edit Meeting Room” (PBI-012) features. I designed the core meeting room fields (including name, area, capacity, location, type, and device) and worked with the project’s general back-end developer, Hanqiu Zhang, to define the API format and data schema, ensuring consistency and compliance of back-end data interfaces. I also maintained close communication with the Booking Module lead, Xinyuan Zhang, to ensure her reservation components (PBI-019/PBI-021) could be built upon the room data foundation I established (Beningo, 2023). During implementation, I encountered technical challenges in supporting image uploads for new rooms. After team discussions and research, I developed a custom UploadImage component (based on Element Plus’s `<el-upload>`), which enabled image upload, preview, and deletion. By the end of Sprint 1, I successfully completed PBI-011 and PBI-012, laying a solid data foundation for subsequent development tasks.

During Sprint 2, I was primarily responsible for implementing two features: “Delete Meeting Room” (PBI-013) and “Room Usage Statistics and Visualization” (PBI-014). For PBI-013, I collaborated closely with Xinyuan Zhang to address the edge cases associated with deleting rooms that have pending bookings. The finalized logic allows administrators to delete meeting rooms even if there are upcoming reservations (i.e., `start_time > current_time`). In such cases, the system retains the associated

booking records while setting the room_id field to null. This design ensures data integrity and avoids cascading deletions, while enhancing operational flexibility. **During the development of PBI-014**, I collaborated with Ziwei Zhou and Jiayi Wang to build the visual analytics module on the system's Main Page dashboard (See [Appendix 5](#)). Using ECharts, we designed three responsive chart panels to present key statistics. Although the “Recent Meetings” and “Latest Announcements” panels were not my assigned PBIs, after discussion in the meeting, I assisted Jiayi Wang in integrating them into the Main Page to improve overall information visibility. During the development of the three <el-card> charts, I initially encountered issues where the chart containers failed to mount in time, causing initialization errors, as well as inconsistent backend data formats that led to rendering problems. To address these, I collaborated with the product owner and backend developers to wrap the chart initialization logic inside Vue 3's nextTick() hook and advocated for standardizing the API response formats. These adjustments ensured seamless data binding with the ECharts components, enabling us to deliver responsive and reliable visualizations on the system dashboard and greatly enhancing administrators' visibility into usage trends. **Additionally**, in the latter phase of this sprint, I supported Ziwei Zhou in debugging the enable/disable functionality for meeting rooms (PBI-016 and PBI-018). Given the close logical relationship between room status management, deletion operations, and the booking workflow, we conducted coordinated integration testing to validate system behavior under various combinations of room states. This effort was critical in ensuring the robustness and correctness of the system in complex usage scenarios (Wild and Lichter, 2023).

Entering Sprint 3, although my assigned PBIs had been largely completed, I continued to participate in system optimization, particularly by assisting Xinrong Shi in developing the front-end meeting room filtering feature (PBI-036 to PBI-040). During the design of the filter, in order to support filtering by room type, capacity, location, and equipment, Xinrong Shi needed to first clarify the standard definitions for these fields (Bell, 2023). Therefore, I closely collaborated with her to provide standardized definitions for relevant meeting room attributes and jointly confirmed the mapping rules between the front-end filter components and the meeting room data fields, ensuring data consistency and accuracy throughout the filtering process. Additionally, I compiled and provided an interface model document to support backend developers in maintaining consistent API specifications. This round of cross-module collaboration helped further unify field application standards and laid a solid foundation for enhancing the system's filtering and query capabilities.

Throughout the three Sprint development cycle, I completed all tasks for the meeting room management module on time and actively collaborated with teammates—including Hanqiu Zhang, Xinyuan Zhang, Jiayi Wang, Ziwei Zhou, and Xinrong Shi—to ensure smooth integration with reservation and filtering features. The structured use of Scrum practices greatly enhanced our team's coordination and development quality while also supporting my own growth across iterations.

Software Design

The key technologies and methodologies used in this project are detailed in the [Appendix 6](#). The database schema design and ER diagram of this project are detailed in [Appendix 7](#). The following section will focus on the development process of two specific PBIs: “Add Meeting Room Information” (PBI-011) and “Meeting Room Usage Statistics and Visualization” (PBI-014).

PBI-011 Add Meeting Room Information

1. Logical Flow (See [Appendix 8](#) for related code)

After logging in with an MRC or Super Admin account, users can navigate to the “Meeting Room

Management” page from the Main Page. There, administrators initiate the room creation process by clicking the “Add” button. To ensure completeness, the system requires input for essential fields including Name, Area, Type, Location, Capacity, and Device; submissions with missing required fields are disallowed. The Image field, while optional at creation, can be supplemented later through the editing function.

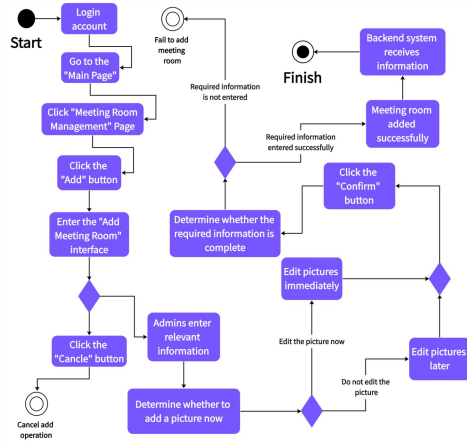


Figure 3-Appendix 9: Activity diagram of PBI-011

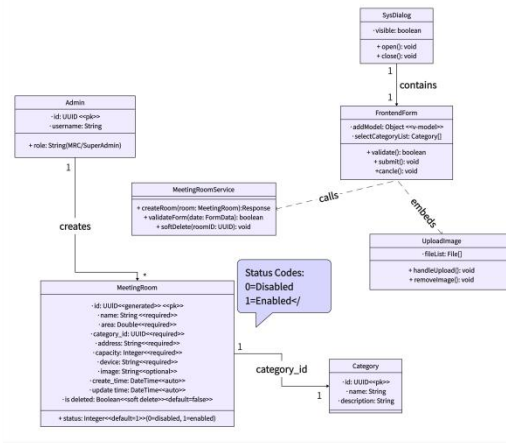


Figure 4-Appendix 10: Class diagram of PBI-011

2. Database Design (See Appendix 11 for related code)

This functionality mainly involves the meeting_room table. Each time a new meeting room is added, a new record is created with a unique id automatically generated by the system. The following fields are populated based on user input: name (room name), area (room area), category_id (room type), address (location), contain (capacity), device (equipment info), and image (image path). Timestamps for create_time and update_time are also recorded. By default, the status field is set to enabled (1), and the disable status is (0), allowing for future enable/disable and logical deletion management. To maintain data consistency and integrity, soft deletion is used (logical deletion instead of physical removal), and UUIDs are used to avoid primary key conflicts (Pettit, 2022).

id # bigint	name varchar(255)	area int	category_id int	address varchar(255)	contain int	device text	image varchar(255)	status tinyint
1	Thought Pavilion	50	4	SIP-North Campus-SD373	12	- Smart collaborative display /	http://47.100.1.155:8089/images/	1
2	Wisdom Chamber	20	6	SIP-North Campus-Area 1 (SE) on FB5F	10	- Mobile group tables and che	http://47.100.1.155:8089/images/	1
3	Inspiration Studio	20	6	SIP-North Campus-Area 2 (NE) on FB5F	10	- Mobile group tables and che	http://47.100.1.155:8089/images/	1
4	Enlightenment Hall	100	7	SIP-North Campus-MA105	100	- HD projector or LED display-	http://47.100.1.155:8089/images/	1
5	Knowledge Hub	100	4	SIP-South Campus-BS4114	50	- Smart collaborative display /	http://47.100.1.155:8089/images/	1
6	Discernment Study	10	5	SIP-North Campus-FB145	6	- Soundproof walls or booth d	http://47.100.1.155:8089/images/	1
7	Diligent Study	120	2	SIP-North Campus-FB251	100	- HD projector / LED video wa	http://47.100.1.155:8089/images/	1
8	Philosophy Room	100	1	SIP-South Campus-HSG26	50	- Large monitor / basic projec	http://47.100.1.155:8089/images/	1
10	Logic Lounge	10	5	SIP-North Campus-FB147	6	- Soundproof walls or booth d	http://47.100.1.155:8089/images/	1
11	Harmony Chamber	150	7	SIP-South Campus-IRG16	200	- HD projector or LED display-	http://47.100.1.155:8089/images/	1
12	Test	100	7	SIP-SA	200	Test	http://47.100.1.155:8089/images/	1

Figure 5-Appendix 12: The Navicat view of the meeting_room table

3. Object State Transitions

Add: When the administrator clicks “Add,” they are directed to the meeting room creation form. The object is in a “draft” state and can be modified or cancelled at any time.

Confirm: After completing the form and clicking “Confirm,” the system validates the data. If valid, it calls the backend API to create a new room entry. The object state is then updated to “active.”

Confirm Error: If the form fails validation (e.g., required fields missing or invalid format), an error

prompt is shown and no data is saved. The user may revise or cancel.

Cancelled: If the administrator cancels or closes the form before submission, the data is discarded and no changes are saved.

4. User Interface Design (See [Appendix 13](#) for related code)

The layout for the Add Meeting Room feature (PBI-011) follows the system's overall UI style. The form is rendered within a SysDialog modal component. Fields such as room name, area, type, location, capacity, equipment info, and image upload are structured using <el-form> and bound to the addModel object using v-model for real-time data binding. The type field uses <el-select> and loads options dynamically via the selectCategoryListApi. Image uploads are handled using a custom UploadImage component, which wraps Element Plus' s <el-upload> to support preview and deletion. Upon clicking confirm, the commit() function is triggered, validating form data (addRef.value?.validate()) and submitting to the backend via addApi. Any form errors or API issues are communicated through Element Plus message components, improving user feedback and interaction quality.

PBI-014 Meeting Room Usage Statistics and Visualization

1. Logical Flow (See [Appendix 14](#) for related code)

After logging in as an MRC or Super Admin, the system automatically displays a set of statistics on the homepage that are closely related to meeting room usage. It first presents an overview of key metrics—including the total number of members, departments, meeting rooms, and pending repairs—to help users quickly grasp the scale of system resources. This is followed by three visual charts showing department member distribution, popular meeting rooms, and the composition of meeting room types, which are generated based on aggregated analysis of recent booking records. By integrating informative summaries with graphical insights, the homepage provides users with a holistic understanding of system operations, thereby supporting more informed resource management and administrative decision-making.

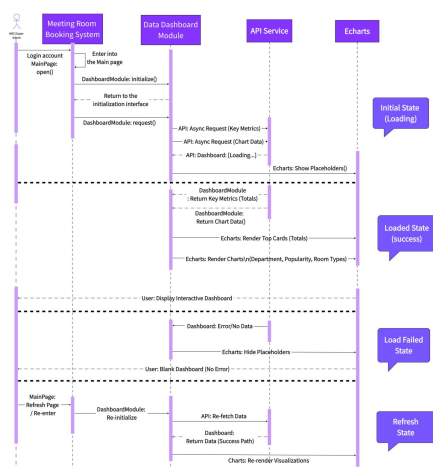


Figure 6-Appendix 15: Sequence diagram of PBI-014

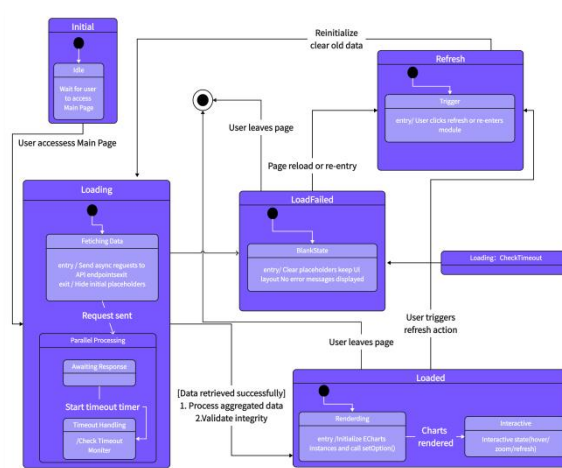


Figure 7-Appendix 16: State Machine diagram of PBI-014

2. Database Design (See [Appendix 17](#) for related code)

This functionality involves five core database tables: sys_user (system users), dept (department information), meeting_room (meeting room information), meeting_book (reservation records), and repair (repair requests). The system dashboard utilizes these tables to display real-time statistics such as

the total number of users, departments, meeting rooms, and pending repairs. Additionally, the system performs multi-dimensional aggregation based on the meeting_book table: it summarizes departmental usage by joining with the dept_id field in sys_user, calculates popular meeting rooms by counting entries grouped by room_id, and analyzes booking distribution by room type through the category_id field in meeting_room. These data are encapsulated and served through backend services such as MeetingBookService, UserService, and RepairService, and are rendered on the frontend using the ECharts library. To ensure performance, most of the aggregation is processed at the SQL level using GROUP BY and COUNT functions, allowing for efficient data retrieval and consistent presentation.

id	room_id	dept_id	user_id	start_time	end_time	status	create_time	use_status	use_desc	apply_desc	process_user	meeting_name
# bigint	# int	# int	# int	datetime	datetime	varchar	datetime	varchar(64)	varchar(255)	varchar(255)	varchar(255)	varchar(255)
17	1	4	14	2025-03-18 23:00:00	2025-03-19 00:00:00	3	2025-03-27 17:46:33	4	Group project discussion	Approved	Administrator	Startup Brainstorming
19	5	4	14	2025-03-27 22:00:00	2025-03-28 00:00:00	1	2025-03-27 17:50:46	1	Startup competition preparation	Approved	Conference Coordinator 1	Business Pitch Rehearsal
20	2	4	10	2025-03-06 04:00:00	2025-03-06 06:00:00	1	2025-03-27 19:59:11	1	Group 17 Weekly Discussion	Approved	Conference Coordinator 1	Group Assignment Discussion
21	3	4	10	2025-03-13 04:00:00	2025-03-13 06:00:00	1	2025-03-27 20:01:35	1	Group 17 Weekly Discussion	Approved	Conference Coordinator 1	Group Assignment Discussion
22	4	1	18	2025-03-26 22:00:00	2025-03-27 00:00:00	1	2025-03-27 21:01:16	1	Extracurricular math learning	Approved	Conference Coordinator 2	Math Club Workshop 1
23	4	6	18	2025-04-02 22:00:00	2025-04-03 00:00:00	1	2025-03-27 21:02:40	1	Extracurricular math learning	Approved	Conference Coordinator 2	Math Club Workshop 2
24	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 21:12:29	0	Self-directed learning	(Null)	(Null)	Self-directed learning
25	10	9	16	2025-04-22 23:00:00	2025-04-22 01:00:00	0	2025-04-21 21:13:46	0	Self-directed learning	(Null)	(Null)	Self-directed learning
26	8	9	21	2025-03-13 18:00:00	2025-03-13 19:00:00	2	2025-03-28 00:52:18	2	Student union activity	Rejected	Conference Coordinator 1	Club Meeting
29	5	2	12	2025-04-01 08:00:00	2025-04-02 08:00:00	1	2025-04-17 23:53:00	1	For discussion	agree	Conference Coordinator 1	CP202 Discussion
30	6	9	16	2025-04-13 03:00:00	2025-04-13 06:00:00	0	2025-04-14 21:12:29	0	Self-directed learning		Conference Coordinator 1	Self-directed learning
31	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
32	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
33	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
34	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
35	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
36	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
37	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
38	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
39	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
40	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
41	1	4	13	2025-04-18 23:00:00	2025-04-19 00:00:00	0	2025-04-19 02:16:13	0	Group Discussion	(Null)	(Null)	Group Discussion
43	1	4	13	2025-04-21 22:00:00	2025-04-21 23:00:00	1	2025-04-20 20:31:54	1	Test1	Agree	Administrator	Test1
44	1	4	11	2025-04-23 22:00:00	2025-04-24 06:00:00	0	2025-04-20 21:15:01	0	Test2	(Null)	Conference Coordinator 1	Test2
46	1	4	13	2025-04-22 22:00:00	2025-04-23 01:00:00	1	2025-04-21 17:37:02	1	test	ok	Administrator	test
47	1	4	44	2025-04-24 22:00:00	2025-04-25 01:00:00	1	2025-04-23 23:01:26	1	1111	1111	Administrator	111

Figure 8-Appendix 18: The Navicat view of the meeting_book table

3. Object State Transitions

Initial: When users access the Main Page, the dashboard module automatically initiates a series of asynchronous requests to fetch statistical data. During this phase, visual components remain in a loading state, awaiting backend responses.

Loaded: Once the system successfully retrieves the statistical data, it dynamically initializes the ECharts instances to render both the top-level info cards—displaying total users, departments, meeting rooms, and pending repairs—and the visual charts below. At this point, users can interactively view key insights such as departmental distribution, meeting room popularity, and booking ratios by room type.

Load Failed: If the backend returns no data or encounters an exception, the system defaults to a blank dashboard. No explicit error message is displayed, but the absence of rendered charts signals potential backend unavailability or data insufficiency.

Refresh: Users can refresh the page or re-enter the module, which re-triggers the entire data loading and rendering process, restoring the dashboard’s interactive state when data is successfully retrieved.

4. User Interface Design (See [Appendix 19](#) for related code)

The usage statistics and visualization module (PBI-014) is embedded in the Main Page to provide administrators with real-time insights into system usage. The dashboard consists of three sections, of which I was primarily responsible for the top and bottom sections: a top section displaying key aggregate metrics—total users, departments, meeting rooms, and pending repairs—rendered using <el-row> and <el-col> components for responsive layout; and a bottom section comprising three ECharts visualizations. These include a bar chart of department member counts (myChart), a horizontal bar chart of popular meeting rooms (myChart1), and a donut chart showing booking distribution by room type (myChart2). The frontend asynchronously requests data from /api/home/getTotal, /getEcharts, /getLastMeeting, and /getCategoryBook. Upon receiving responses, charts are initialized

using `global.$charts.init()` and configured via `setOption()`. To ensure visual consistency and robustness, the rendering is deferred until the DOM is mounted using Vue 3's `nextTick()` lifecycle hook. Axis labels are auto-rotated or truncated for clarity, and a unified purple color palette enhances visual coherence. In the event of data loading failures, the system omits error messages and leaves the chart areas blank to preserve the clean layout. This layered visualization design improves administrative awareness of operational trends and provides data support for strategic decisions (Wood, 2019).

Change Management

In Scrum projects, requirements often evolve throughout the development cycle. To manage these changes effectively, our team strictly adhered to Scrum principles: we captured and discussed changes promptly during Sprint planning meetings and kept our shared Sprint tracking documents updated accordingly. While ensuring both quality and development efficiency, we flexibly integrated changes into ongoing development tasks. Within the meeting room management module I was responsible for (PBI-011 to PBI-014), there were two typical examples of change management.

1. Add Meeting Room (PBI-011): Integration of Image Upload Function

In the initial requirement design, PBI-011 (Add Meeting Room Information) only required administrators to fill in basic attributes of the meeting room, such as name, area, location, category, capacity, and device information. The image upload function was originally planned as a separate PBI to be developed in a later stage. However, during the Sprint 1 review meeting, the team concluded that asking administrators to upload images separately after the fact would disrupt the workflow and increase system maintenance complexity. Therefore, after confirming with the product owner and both frontend and backend developers, we decided to integrate the image upload feature into PBI-011 ahead of schedule. I updated the frontend interface by embedding the `UploadImage` component into the meeting room creation form (`SysDialog` modal), and modified the backend API to support direct uploading and saving of image paths during the room creation process. This change was successfully completed within the same Sprint and did not impact the overall delivery timeline. The adjustment significantly improved feature cohesion and user experience while maintaining data consistency.

2. Adjustment to Meeting Room Deletion Logic (PBI-013)

In the initial design of the project, the deletion function for meeting rooms (PBI-013) was intended to include a reservation check mechanism — that is, if a meeting room had future bookings (where `start_time > current_time`), deletion would be blocked in order to maintain data consistency. However, during actual development and Sprint review meetings, the team reconsidered this approach based on system simplicity and practical business needs, and revised the logic. The updated rule allows administrators to delete a meeting room even if it has future reservations. These deleted rooms no longer affect the integrity of existing booking records, although the “Room Name” field in the Meeting Room Booking page will appear blank for those records. This design retains users’ historical booking data while avoiding deletion failures caused by foreign key dependencies, thereby improving system flexibility and fault tolerance.

Legal, social, ethical, and professional issues

During the development of the meeting room booking system, we placed strong emphasis on legal compliance, social responsibility, ethical standards, and professional practices. As the system targets internal organizational users and involves public resource management and user data collection, the

modules I was responsible for (PBI-011 to PBI-014) gave particular attention to related issues. **From a legal perspective**, the system collects users' personal information such as names and departments for reservation management purposes. These types of data fall under the protection scope of laws like the Personal Information Protection Law (Ma and Shi, 2024). To address this, the system clearly informs users of the purpose and scope of data collection during registration, prohibits the use of such data for commercial or external purposes, and enforces access control and encryption for sensitive database fields. Regarding the deletion of meeting rooms, if related booking records exist, the "Room Name" field is displayed as blank. To avoid concerns about data integrity, the system will not delete records in the user interface and will retain the original room information in the backend log for traceability. **From a social standpoint**, the fair use of public resources is a critical consideration. The system introduces a role-based approval mechanism (e.g., MRC Coordinator review) to ensure that booking processes remain transparent and open. In addition, through the booking statistics feature (PBI-014), the system presents overall departmental usage trends to discourage resource allocation bias and promote fairness within the organization. **From an ethical standpoint**, the collection and display of departmental reservation data must be handled with care. The system does not rank or publicly compare departments but instead focuses on overall trends. It is also clearly stated that this data is used strictly for internal optimization purposes, helping to avoid misuse for performance evaluation or internal competition, thereby minimizing ethical risks. **From a professional perspective**, while developing the meeting room management functionality, it was essential to ensure that the system could correctly process user operations and provide clear feedback at key stages. Logical checks and confirmation prompts were implemented before major operations to guide users effectively. This aligns with the IEEE Code of Ethics on providing reliable service and the ACM Code of Ethics regarding maintaining high professional standards (Brinkman *et al.*, 2016).

Conclusion

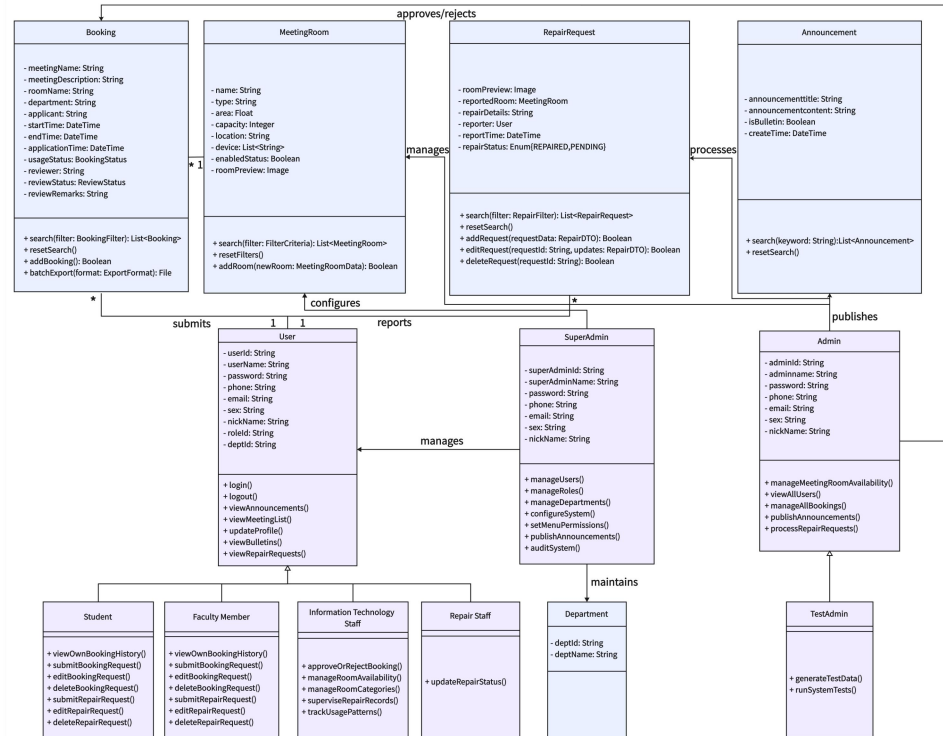
Through my participation in the development of the meeting room booking system, I gained a more systematic understanding of the software development process. Adopting the Scrum agile methodology allowed us to respond quickly to changing requirements and continuously improve the project through regular Sprint planning meetings and reviews. Although the team initially lacked a deep understanding of Scrum, which led to some delays in development, we gradually mastered and refined our collaboration practices through hands-on experience. On a personal level, I was mainly responsible for the meeting room management module (PBI-011 to PBI-014), including front-end interface design, feature implementation, and data visualization. Throughout this process, I significantly improved my programming skills, system integration capabilities, and user experience design awareness. Of course, there were still some limitations in the project. The current system is only compatible with desktop platforms and lacks mobile adaptation. The approval process relies heavily on manual operation, which lowers efficiency. The role-based access control is relatively rigid, making it less flexible for future feature expansion. Additionally, our test coverage was insufficient, leading to issues being discovered only after deployment. Moving forward, we should enhance responsive design, incorporate automated approval workflows, and strengthen the testing framework. From a professional perspective, I also became more attentive to legal compliance and user privacy. In future projects, I will actively follow accessibility standards such as the WCAG to improve the system's inclusiveness and reliability. This development experience not only enhanced my technical and collaboration abilities but also laid a solid foundation for my future growth as a professional software engineer.

References (if needed. Not count into the max page limit)

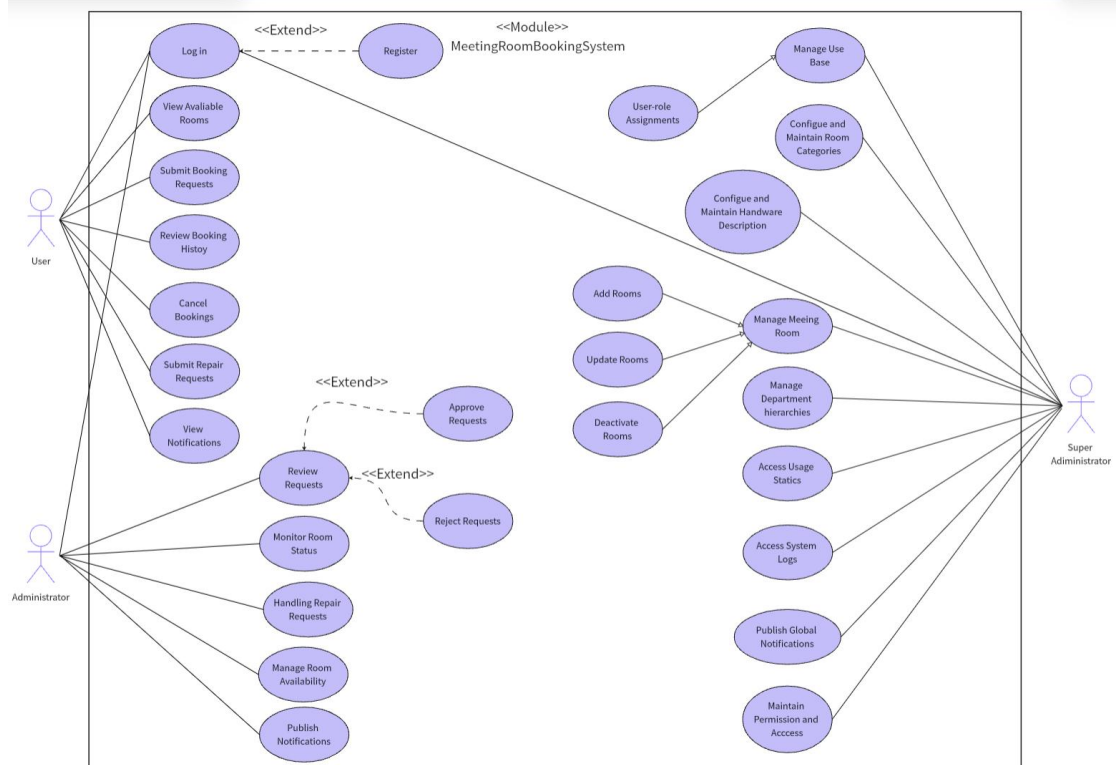
- Bell, C.A. (2023) *MySQL database service revealed: running MySQL as a service in the Oracle Cloud Infrastructure*. Apress. Available at:
<https://search-ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0002346770&site=eds-live&scope=site> (Accessed: 19 May 2025).
- Beningo, J. (2023) *Embedded software design: a practical approach to architecture, processes, and coding techniques*. Apress. Available at:
<https://search-ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0002313901&site=eds-live&scope=site> (Accessed: 19 May 2025).
- Brinkman, B., *et al.* (2016) ‘Making a Positive Impact: Updating the ACM Code of Ethics’ , *Communications of the ACM*, 59(12), pp. 7 – 13. doi:10.1145/3015149.
- Georgeta, S., *et al.* (2018) ‘A Smart Meeting Room Scheduling and Real-Time Management System of Occupancy Detection to Ford Romania’ , *International Multidisciplinary Scientific Conference on Social Sciences & Arts SGEM*, 5, pp. 3 – 10. doi:10.5593/sgemsocial2018/1.5.
- Ma, X. and Shi, X. (2024) ‘Privacy protection and corporate performance: Empirical evidence based on the Personal Information Protection Law’ , *Economic Science / Jingji Kexue*, (6), pp. 122 – 147. doi:10.12088/PKU.jjxk.2024.06.06.
- Pettit, T. (2022) *The MySQL workshop: a practical guide to working with data and managing databases with MySQL*. Edited by S. Cosentino. Packt. Available at:
<https://search-ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0002100750&site=eds-live&scope=site> (Accessed: 19 May 2025).
- Sutherland, J.V. (2014) *Scrum: the art of doing twice the work in half the time*. First Edition. Crown Business. Available at:
<https://search-ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0000465521&site=eds-live&scope=site> (Accessed: 19 May 2025).
- Wild, N. and Lichter, H. (2023) ‘Unit Test Based Component Integration Testing’ , *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 1 – 10. doi:10.1109/APSEC60848.2023.00010.
- Wood, D. (2019) *Interface design: an introduction to visual communication in UI design*. Bloomsbury Visual Arts (Basics interactive design). Available at:
<https://search-ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0001495986&site=eds-live&scope=site> (Accessed: 19 May 2025).

Appendix

Appendix 1: System module design class diagram



Appendix 2: Use case diagram of the system



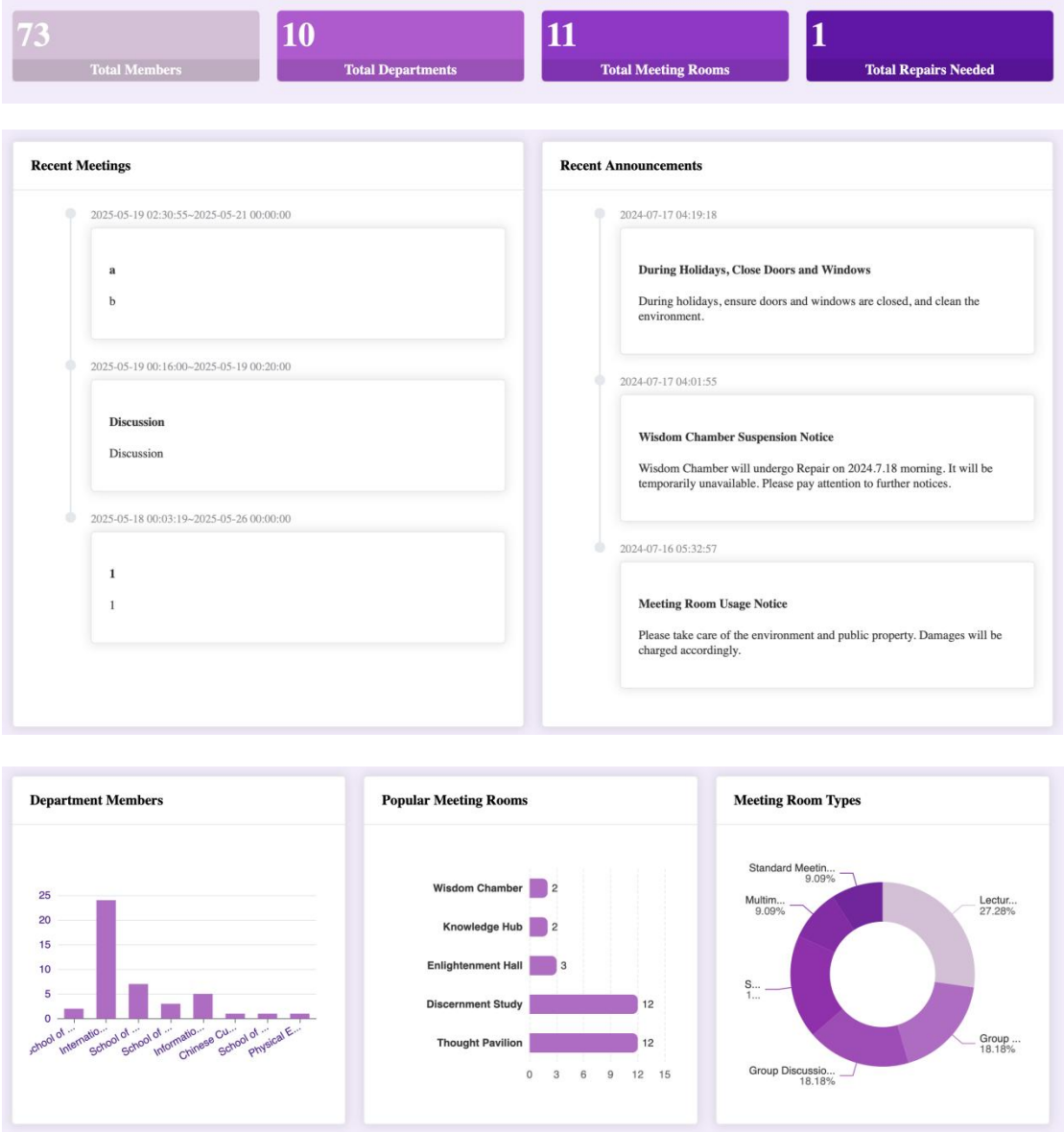
Appendix 3: Scrum methodology

Person	Collaborator	Collaborative PBIs (ID + Description)	Reason and Collaboration Details
Zimo Zheng (User Account Management)	Hanqiu Zhang (Role & Permission Management)	-PBI-001: Register Account -PBI-003: Email & Password Login -PBI-006: Support User Roles (Admin / Regular User)	Hanqiu can assign role permissions only after Zimo has implemented account registration and login. They need to coordinate on user table structure changes and role field integration.
Hanqiu Zhang (Role & Permission Management)	Jiayi Wang (Approval Process)	-PBI-007: Regular Users Can Only Book Rooms -PBI-008: Modify User Role -PBI-026: Admin View All Reservations	Jiayi's approval features rely on Hanqiu's role management implementation, as only admins can batch approve and view all bookings. Close coordination is required on user role validation APIs.
Nan Lin (Meeting Room Management)	Xinyuan Zhang (Reservation Flow)	-PBI-011: Add Meeting Room -PBI-012: Edit Meeting Room -PBI-019: View Future Room Availability -PBI-021: View Room List	Xinyuan's reservation module must rely on the meeting room info built by Nan . They must align on field designs (e.g., room status, capacity, location) and API endpoints.
Nan Lin (Meeting Room Management)	Ziwei Zhou (Enable/Disable + Email Alerts)	-PBI-016: Mark Room as Enabled/Disabled -PBI-018: Block Reservations for Disabled Rooms	Ziwei can only manage the enable/disable status after Nan creates the rooms. They must agree on status field definitions (e.g., <i>enabled = true/false</i>).
Xinyuan Zhang (Reservation Flow)	Jiayi Wang (Approval Management)	-PBI-024: Fill in Reservation Info -PBI-025: Cancel Reservation -PBI-026: View Reservation History -PBI-027: Approve Reservation Requests	Jiayi's approval system must be built upon the reservation functionality developed by Xinyuan . They must agree on the data format and time fields of reservation records.
Jiayi Wang (Approval Management)	Xiaoya Li (Rules & Restrictions)	-PBI-027: Approve Reservation Requests -PBI-031-035: Timeouts, Bans, Unlocks	Xiaoya's logic for time and quota constraints must act upon reservation actions that occur after Jiayi's approval process. For instance, rejecting users who exceed booking limits.
Xiaoya Li (Rules & Restrictions)	Xinrong Shi (Filtering & History Records)	-PBI-036-040: Search Filters + History Viewing -PBI-031-035: Restrictions	For filtering and history logic to function properly, Xiaoya's user restriction logic must be finalized first. Xinrong's frontend filters will then integrate those restrictions.
Ziwei Zhou (Enable/Disable + Email)	Xinyuan Zhang (Reservation Flow)	-PBI-018: Block Reservation for Disabled Rooms -PBI-041: Email Notification of Reservation Results	Email alerts rely on the success/failure of reservations handled by Xinyuan . They need to define the trigger mechanism for status changes that fire email notifications.
Xinrong Shi (Filtering & History)	Nan Lin (Meeting Room Management)	-PBI-036-040: Search Filters (Capacity, Location, Equipment)	When building the filter system, Xinrong must refer to Nan's field definitions for meeting rooms, such as <i>contain (capacity)</i> , <i>add_location</i> , and <i>device</i> . They need to agree on field naming conventions.
Xiangyi Zhu (Email Notification & History)	Jiayi Wang (Approval Management) Ziwei Zhou (Disable Reservations)	-PBI-041: Reminder for Past 7-Day Reservation Count -PBI-045: Users Check Approval Status on "My Reservations"	The reminder feature (PBI-041) relies on Jiayi's approval data, as status changes trigger notifications. - For reservation failure reminders (PBI-045), Xiangyi must align with Ziwei's enable/disable mechanism to ensure room status accuracy.

Appendix 4: Sprint tracking document

Sprint Progress Report							
Project Name	An online meeting booking system						
Team Number	17						
Team Name	Group17						
Team Leader	Xiangyi.Zhu						
TA	Tong.Zou						
Week	Sprint	PBI	Owener	Start Date	Expected Completion Date	Actual Completion Date	
1	1	User account registration	Zimo Zheng	2/17/2025	2/18/2025	2/18/2025	
	1	Edit personal information after logging in	Zimo Zheng	2/17/2025	2/18/2025	2/18/2025	
	1	Login using email and password	Zimo Zheng	2/17/2025	2/18/2025	2/18/2025	
	1	Role-based access control (Admin / Regular User)	Hanqiu Zhang	2/19/2025	2/21/2025	2/21/2025	
2	1	Regular users can book rooms but cannot manage them	Hanqiu Zhang	2/19/2025	2/21/2025	2/21/2025	
	1	Add meeting room information	Nan Lin	2/24/2025	2/27/2025	2/27/2025	
	1	Edit meeting room information	Nan Lin	2/24/2025	2/27/2025	2/27/2025	
	1	View room availability for the upcoming week	Xinyuan Zhang	2/24/2025	2/27/2025	2/27/2025	
3	2	View meeting room list and detailed information	Xinyuan Zhang	2/24/2025	2/27/2025	2/27/2025	
	2	Secure logout	Zimo Zheng	3/3/2025	3/9/2025	3/9/2025	
	2	Password recovery functionality	Zimo Zheng	3/3/2025	3/9/2025	3/9/2025	
	2	Admin can modify user roles	Hanqiu Zhang	3/3/2025	3/9/2025	3/9/2025	
4	2	Admin can view all users	Hanqiu Zhang	3/3/2025	3/9/2025	3/9/2025	
	2	Delete meeting room information	Nan Lin	3/3/2025	3/9/2025	3/9/2025	
	2	Meeting room usage statistics and analysis	Nan Lin	3/3/2025	3/9/2025	3/9/2025	
	2	Search meeting rooms	Xinyuan Zhang	3/10/2025	3/16/2025	3/16/2025	
5	2	Filter meeting rooms based on specific criteria	Xinyuan Zhang	3/10/2025	3/16/2025	3/16/2025	
	2	Fill in booking details and make a booking	Xinyuan Zhang	3/10/2025	3/16/2025	3/16/2025	
	2	Manage maintenance of meeting room equipment	Ziwei Zhou	3/10/2025	3/16/2025	3/16/2025	
	2	Mark meeting rooms as enabled/disabled	Ziwei Zhou	3/10/2025	3/16/2025	3/16/2025	
6	3	Disable booking for rooms that are marked as disabled	Ziwei Zhou	3/10/2025	3/16/2025	3/16/2025	
	3	Cancel room booking	Xinyuan Zhang	3/17/2025	3/23/2025	3/23/2025	
	3	Admin can view all booking records	Jiayi Wang	3/17/2025	3/23/2025	3/23/2025	
	3	Admin can approve booking requests	Jiayi Wang	3/17/2025	3/23/2025	3/23/2025	
7	3	Admin can reject booking requests (with reason)	Jiayi Wang	3/17/2025	3/23/2025	3/23/2025	
	3	Users can view the status of their own bookings	Jiayi Wang	3/17/2025	3/23/2025	3/23/2025	
	3	Admin can bulk approve bookings	Jiayi Wang	3/17/2025	3/23/2025	3/23/2025	
	3	Booking time restrictions, auto-ban for violations, and manual unlock	Xiaoya Li	3/24/2025	3/30/2025	3/30/2025	
8	3	Enhanced filtering options (by capacity, location, equipment)	Xinrong Shi	3/24/2025	3/30/2025	3/30/2025	
	3	Improved search and detailed room view	Xinrong Shi	3/24/2025	3/30/2025	3/30/2025	
	3	Display user's booking count from the past 7 days during booking	Xiangyi Zhu	3/24/2025	3/30/2025	3/30/2025	
	3	View canceled booking history, usage history, and modification requests	Xiangyi Zhu	3/24/2025	3/30/2025	3/30/2025	
3	3	Allow users to view the current approval status of their booking records in the "Meeting Room Booking" interface	Xiangyi Zhu	3/24/2025	3/30/2025	3/30/2025	

Appendix 5: Visual analytics module on the system ’ s Main Page dashboard



Technical Specification Document

1. Development Tools

- **Version Control:**
 - Git (Code Management)
 - Coding.net (Remote Repository)
- **Project Management:**
 - Coding.net (Scrum Task Management)
 - Wiki (Document Collaboration)
- **Database Management:**
 - Navicat (v11.1.13) – Visual Database Management
- **IDE & Editors:**
 - Visual Studio Code (v1.98.0) – Frontend & Backend Development
- **Documentation & Notes:**
 - Word (Formal Documentation)

2. Programming Languages

- **Frontend:**
 - HTML, CSS (Basic Structure & Styling)
 - JavaScript (Frontend Interaction)
 - TypeScript (Enhanced Code Robustness)
- **Backend:**
 - Java 17 (v17.0.14)

3. Frameworks & Libraries

- **Frontend:**

- Vue 3 (v3.3.4) – Frontend Framework
- Vite (v4.4.5) – Build Tool
- Vue Router (v4.2.4) – Routing
- Element Plus (v2.3.12) – UI Component Library
- Axios (v1.7.2) – Frontend API Request Tool
- ECharts (v5.4.3) – Data Visualization
- FullCalendar + Plugin Bundle (v6.1.15) – Calendar Component
- Vue-Tippy (v6.6.0) – Tooltip Library

- **Backend:**

- Spring Boot (v3.2.7) – Main Framework
- Maven (v3.8.1) – Dependency Management & Packaging
- Druid (v1.2.1) – Database Connection Pool
- MyBatis-Plus + MyBatis-Plus-Join (v3.5.7 + v1.4.7.2) – ORM Framework
- Sa-Token (v1.38.0) – Permission Authentication
- Fastjson (v1.2.68) – JSON Processing Tool
- Spring Security Crypto, JWT (Java-JWT) (v6.1.4 + v3.10.3) – Security Components

- **Database:**

- MySQL (v5.7.27) – Relational Database

- **Authentication:**

- JWT (JSON Web Token)

- **Testing Frameworks:**

- Jest / Mocha – Frontend & Backend Testing
- JUnit – Java Unit Testing

4. Required Knowledge

4.1 Software Engineering & Development Workflow

- **Scrum Agile Development:**

- Sprint Planning, Iterative Development, Daily Standups
- PBI (Product Backlog Item) Management
- Task Breakdown (Epic / Story / Task)

- **Database Design:**

- Database Schema Design (ERD)
- Relational Databases
- SQL Query Optimization (Indexing, Transactions)

4.2 Frontend Technologies

- Vue 3 for Component-based Development
- Vue Router for Page Navigation
- Axios for HTTP Requests
- ECharts for Data Visualization
- Element Plus for Form Validation & Modal Windows

4. Required Knowledge

4.1 Software Engineering & Development Workflow

- **Scrum Agile Development:**

- Sprint Planning, Iterative Development, Daily Standups
- PBI (Product Backlog Item) Management
- Task Breakdown (Epic / Story / Task)

- **Database Design:**

- Database Schema Design (ERD)
- Relational Databases
- SQL Query Optimization (Indexing, Transactions)

4.2 Frontend Technologies

- Vue 3 for Component-based Development
- Vue Router for Page Navigation
- Axios for HTTP Requests
- ECharts for Data Visualization
- Element Plus for Form Validation & Modal Windows

5. Suggested Workflow

5.1 Project Initialization

- Initialize Frontend & Backend with Vue + Vite + Spring Boot
- Set up Git Project Structure: `main` / `develop` / `student ID` branches
- Configure Unified API Response Format and Exception Handling Mechanism

5.2 Requirements Analysis & Task Allocation

- Record Requirements Using Wiki
- Assign Tasks via Coding.net
- Build ERD Based on User Stories
- Design Functional Modules, UI Prototypes, and Role Permission Hierarchy

5.3 Development Phase

- **Frontend:** API Integration, Database Binding, Validation Logic
- **Backend:** Interface Logic, API Endpoints, State Management

5.4 Testing & Integration

- Conduct Integration Testing, Monitor User Behavior via Sentry

5.5 Deployment

- **Backend:** Package into `.jar` File
- **Frontend:** Build via Vite and Upload to Server

Appendix 7: The database schema design and ER diagram of this project

Database Table Structure Documentation

Database Name: meeting-room-system

Version: 1.0.0

Description: Database Design Documentation

Table:meeting_category (Meeting Room Type)

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	id	bigint	20	0	N	Y		Serial number
2	category_name	varchar	255	0	Y	N		Category name
3	order_num	varchar	255	0	Y	N		Category order No.

Table:meeting_room (Meeting Room Table)

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	id	bigint	20	0	N	Y		Serial number
2	name	varchar	255	0	Y	N		Meeting room name
3	area	int	10	0	Y	N		Area
4	category_id	int	10	0	Y	N		Meeting room category
5	address	varchar	255	0	Y	N		Address
6	contain	int	10	0	Y	N		Capacity
7	device	text	65535	0	Y	N		Device
8	image	varchar	255	0	Y	N		Meeting room images
9	status	tinyint	0	0	Y	N		Enabled status

Table:meeting_book (Appointment Management)

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	id	bigint	20	0	N	Y		Serial number
2	room_id	int	10	0	Y	N		Meeting room name
3	dept_id	int	10	0	Y	N		User department
4	user_id	int	10	0	Y	N		User id

5	start_time	datetime	19	0	Y	N		Start time
6	end_time	datetime	19	0	Y	N		End time
7	status	varchar	64	0	Y	N		Audit status
8	create_time	datetime	19	0	Y	N		Create time
9	use_status	varchar	64	0	Y	N		Use status

Table : repair (Repair Management)

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	id	bigint	20	0	N	Y		Serial number
2	title	varchar	255	0	Y	N		Title
3	content_text	longtext	2147483647	0	Y	N		Repair details
4	user_id	int	10	0	Y	N		Person who reports a repair
5	create_time	datetime	19	0	Y	N		Time to report for repair
6	image	varchar	255	0	Y	N		Images of reported repairs
7	room_id	int	10	0	Y	N		Meeting room reported for repairing

Table: sys_dept (Department Table)

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	id	bigint	20	0	N	Y		Serial number
2	dept_name	varchar	64	0	Y	N		Department name
3	order_num	int	10	0	Y	N		Department number

Table : sys_menu

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	menu_id	int	10	0	N	Y		Menu id
2	parent_id	int	10	0	Y	N		Parent id
3	title	varchar	36	0	Y	N		Menu title
4	code	varchar	36	0	Y	N		Permission field
5	name	varchar	36	0	Y	N		Route name
6	path	varchar	36	0	Y	N		Routing address

7	url	varchar	128	0	Y	N		Component path
8	type	varchar	2	0	Y	N		Menu type 0:Catalogue 1:Menu 2:Button
9	icon	varchar	36	0	Y	N		Menu icon
10	parent_name	varchar	36	0	Y	N		Parent menu name
11	order_num	int	10	0	Y	N		Serial number
12	create_time	datetime	19	0	Y	N		Create time
13	update_time	datetime	19	0	Y	N		Update time

Table: **sys_notice** (Announcement Table)

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	id	bigint	20	0	N	Y		Serial number
2	title	varchar	255	0	Y	N		Announcement title
3	notice_text	longtext	2147483647	0	Y	N		Announcement
4	create_time	datetime	19	0	Y	N		Create time

Table: **sys_role**

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	role_id	int	10	0	N	Y		Role id
2	role_name	varchar	36	0	Y	N		Role name
3	type	varchar	4	0	Y	N		Type
4	remark	varchar	64	0	Y	N		Description
5	create_time	datetime	19	0	Y	N		Create time
6	update_time	datetime	19	0	Y	N		Update time

Table : **sys_role_menu**

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	role_menu_id	int	10	0	N	Y		Primary key
2	role_id	int	10	0	Y	N		Role id
3	menu_id	int	10	0	Y	N		Menu id

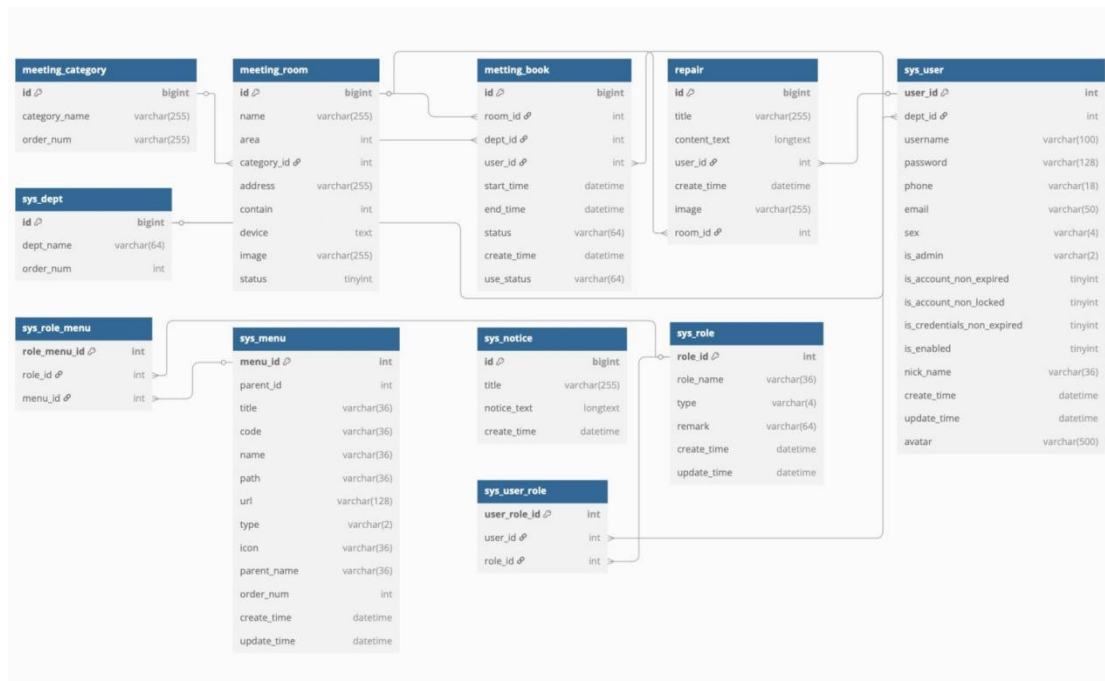
Table : sys_user

No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	user_id	int	10	0	N	Y		User id
2	dept_id	int	10	0	Y	N		Department id
3	username	varchar	100	0	Y	N		Login account
4	password	varchar	128	0	Y	N		Login password
5	phone	varchar	18	0	Y	N		Telephone number
6	email	varchar	50	0	Y	N		Email address
7	sex	varchar	4	0	Y	N		Gender
8	is_admin	varchar	2	0	Y	N		Whether super administrator 1: Yes 0: No
9	is_account_non_expired	tinyint	4	0	Y	N		Whether the account is expired 1 : Not expired 0 : Expired
10	is_account_non_locked	tinyint	4	0	Y	N		Whether the account is locked 1 : unlocked 0 : locked
11	is_credentials_non_expired	tinyint	4	0	Y	N		Whether the password has expired 1 : not expired 0 : expired
12	is_enabled	tinyint	4	0	Y	N		Whether the account is available 1 : available 0 : removes the user
13	nick_name	varchar	36	0	Y	N		Nick name
14	create_time	datetime	19	0	Y	N		Create time

15	update_time	datetime	19	0	Y	N		Update time
16	avatar	varchar	500	0	Y	N		Avatar path

Table : sys_user_role

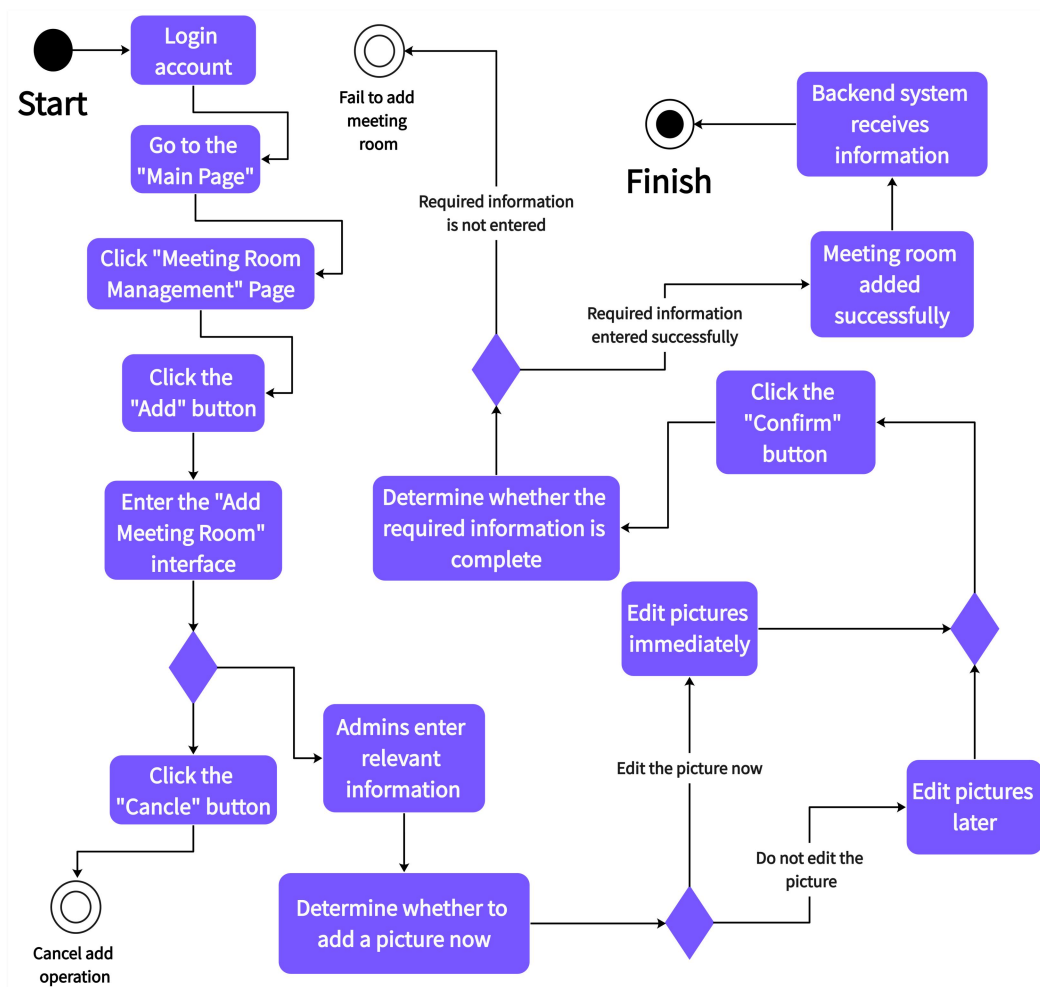
No.	Field Name	Data Type	Length	Decimal	Nullable	Primary Key	Default	Description
1	user_role_id	int	10	0	N	Y		Primary key
2	user_id	int	10	0	Y	N		User id
3	role_id	int	10	0	Y	N		Role id



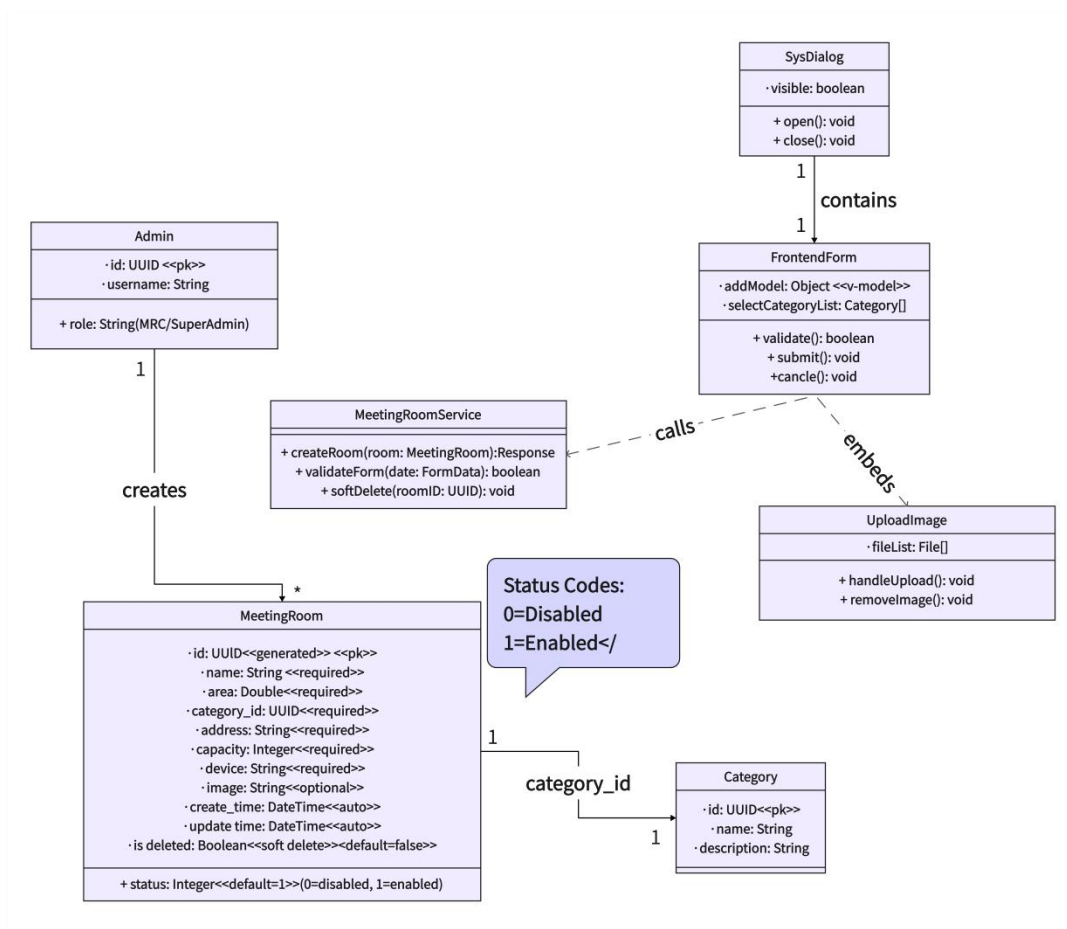
Appendix 8: PBI-011 Add Meeting Room Information

```
/**
 * Newly added
 *
 * @param meetingRoom
 * @return
 */
@PostMapping(Ⓜ "add")
@SaCheckPermission("sys:meetingRoom:add")
public ResultVo add(@RequestBody MeetingRoom meetingRoom) {
    if (meetingRoomService.save(meetingRoom)) {
        return ResultUtils.success(msg: "Added successfully");
    }
    return ResultUtils.error(msg: "Failed to add!");
}
```

Appendix 9: Activity diagram of PBI-011



Appendix 10: Class diagram of PBI-011



Appendix 11: PBI-011 Add Meeting Room Information Database Design

```
public class MeetingRoom implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * Serial Number
     */
    @TableId(value = "id", type = IdType.AUTO)
    @ExcelProperty("Serial Number")
    @ExcelIgnore
    private Long id;

    /**
     * Meeting Room Name
     */
    @TableField("name")
    @ExcelProperty("Meeting Room Name")
    @ColumnWidth(20)
    private String name;

    @TableField(exist = false)
    @ExcelProperty("Meeting room type")
    @ColumnWidth(20)
    private String categoryName;

    /**
     * Area
     */
    @TableField("area")
    @ExcelProperty("area")
    private Integer area;
}
```

Appendix 12: The Navicat view of the meeting_room table

id # bigint	name # varchar(255)	area # int	category_id # int	address # varchar(255)	contain # int	device # text	image # varchar(255)	status # tinyint
1	Thought Pavilion	50	4	SIP-North Campus-SD373	12	- Smart collaborative display /	http://47.100.1.155:8089/images/	1
2	Wisdom Chamber	20	6	SIP-North Campus-Area 1 (SE) on FB5F	10	- Mobile group tables and chairs	http://47.100.1.155:8089/images/	1
3	Inspiration Studio	20	6	SIP-North Campus-Area 2 (NE) on FB5F	10	- Mobile group tables and chairs	http://47.100.1.155:8089/images/	1
4	Enlightenment Hall	100	7	SIP-North Campus-MA105	100	- HD projector or LED display-	http://47.100.1.155:8089/images/	1
5	Knowledge Hub	100	4	SIP-South Campus-BS4114	50	- Smart collaborative display /	http://47.100.1.155:8089/images/	1
6	Discernment Study	10	5	SIP-North Campus-FB145	6	- Soundproof walls or booth d	http://47.100.1.155:8089/images/	1
7	Diligent Study	120	2	SIP-North Campus-FB251	100	- HD projector / LED video wa	http://47.100.1.155:8089/images/	1
8	Philosophy Room	100	1	SIP-South Campus-HSG26	50	- Large monitor / basic projec	http://47.100.1.155:8089/images/	1
10	Logic Lounge	10	5	SIP-North Campus-FB147	6	- Soundproof walls or booth d	http://47.100.1.155:8089/images/	1
11	Harmony Chamber	150	7	SIP-South Campus-IRG16	200	- HD projector or LED display-	http://47.100.1.155:8089/images/	1
12	Test	100	7	SIP-SA	200	Test	http://47.100.1.155:8089/images/	1

Appendix 13: User Interface Design of PBI-011 Add Meeting Room Information

```
<template>
  <SysDialog
    <template v-slot:content>
      <el-form
        >
          <el-row>
            <el-col :span="20" :offset="2"> <!-- Center align the left side -->
              <el-form-item label="Name:" prop="name">
                <el-input v-model="addModel.name" />
              </el-form-item>
              <el-form-item label="Area:" prop="area">
                <el-input v-model="addModel.area" />
              </el-form-item>
              <el-form-item label="Type:" prop="categoryId">
                <el-select
                  v-model="addModel.categoryId"
                  clearable
                  placeholder="Please select a room type"
                  style="width: 100%"
                >
                  <el-option
                    v-for="item in options"
                    :key="item['value']"
                    :label="item['label']"
                    :value="item['value']"
                  />
                </el-select>
              </el-form-item>
              <el-form-item label="Location:" prop="address">
                <el-input v-model="addModel.address" />
              </el-form-item>
              <el-form-item label="Capacity:" prop="contain">
                <el-input v-model="addModel.contain" />
              </el-form-item>
              <el-form-item label="Device:" prop="device">
                <el-input
                  :rows="5"
                  type="textarea"
                  v-model="addModel.device"
                ></el-input>
              </el-form-item>
              <el-form-item prop="image" label="Image:">
                <UploadImage
                  ref="upImgRef"
                  @getImg="getImg"
                  :oldUrl="oldUrl"
                />
              </el-form-item>
            </el-col>
          </el-row>
        </el-form>
      </template>
    </SysDialog>
  </template>
```

```
import http from "@http";
// Data model
export type MeetingRoom = {
  id: string,
  name: string,
  area: string,
  categoryId: string,
  address: string,
  contain: string,
  device: string,
  image: string,
}
// Get list params
export type MeetingRoomPageParm = {
  currentPage: number,
  pageSize: number,
  total: number,
  name: string
}
// Add
export const addApi = (parm: MeetingRoom) => {
  return http.post("/api/meetingRoom/add", parm)
}
```

```
// Add
const addBtn = () => {
  selectCategoryList();
  // Clear image data
  fileList.value = [];
  oldUrl.value = [];
  addModel.id = "";
  tags.value = "0";
  dialog.title = "Add";
  dialog.width = 900;
  dialog.height = 450;
  // Show dialog
  onShow();
  // Clear form
  addRef.value?.resetFields();
};
```

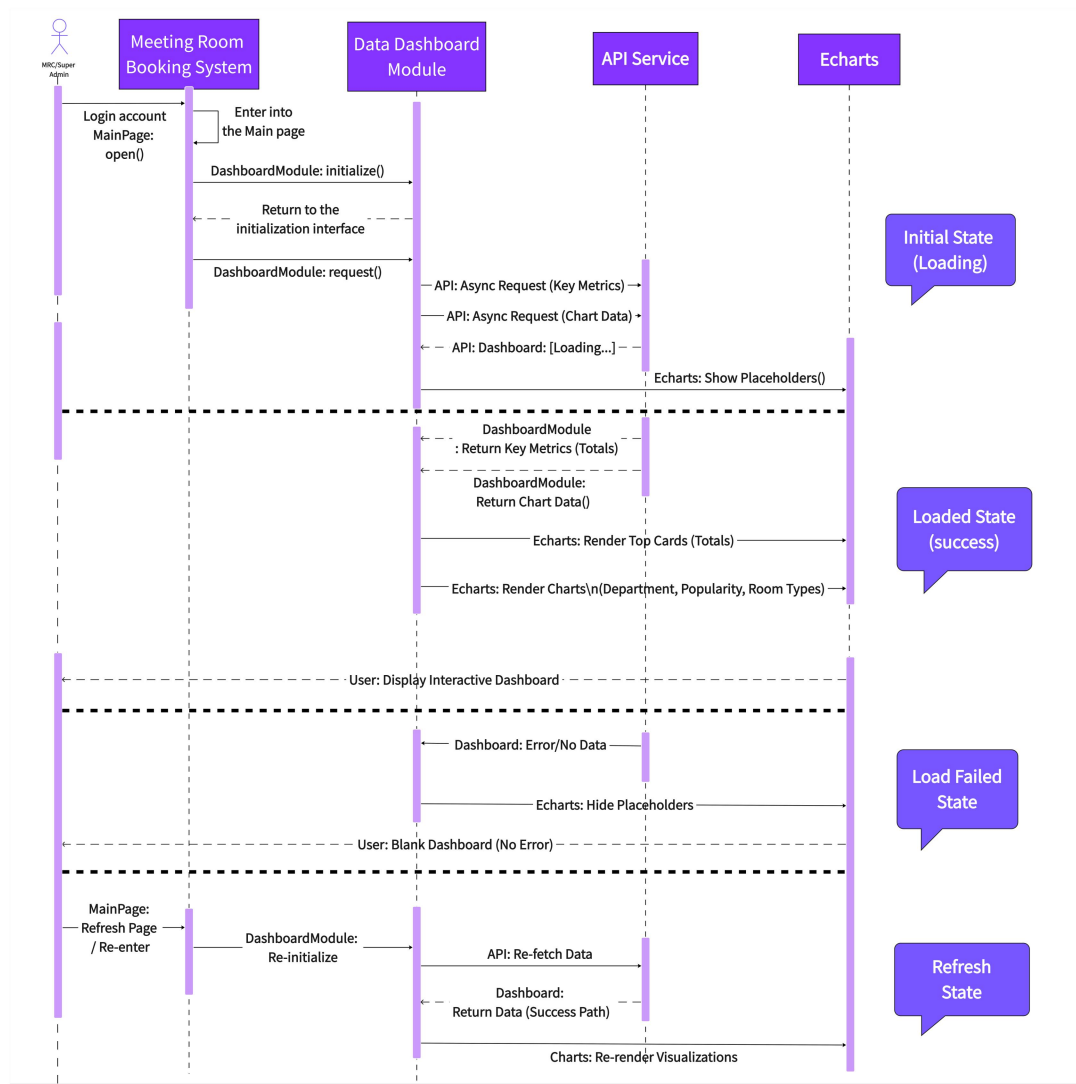
Appendix 14: PBI-014 Meeting Room Usage Statistics and Visualization

```
/**
 * Histogram: Query the total number of employees in each department.
 * @return
 */
@GetMapping(#{@"/getEcharts"})
public ResultVo getEcharts(){
  List<EchartItem> echarts = deptService.getEcharts();
  Echarts vo = new Echarts();
  if(echarts.size() > 0){
    List<String> names = new ArrayList<>();
    List<Integer> values = new ArrayList<>();
    for (int i=0;i<echarts.size();i++){
      names.add(echarts.get(i).getDeptName());
      values.add(echarts.get(i).getValue());
    }
    vo.setNames(names);
    vo.setValues(values);
  }
  return ResultUtils.success( msg: "Query successful",vo);
}

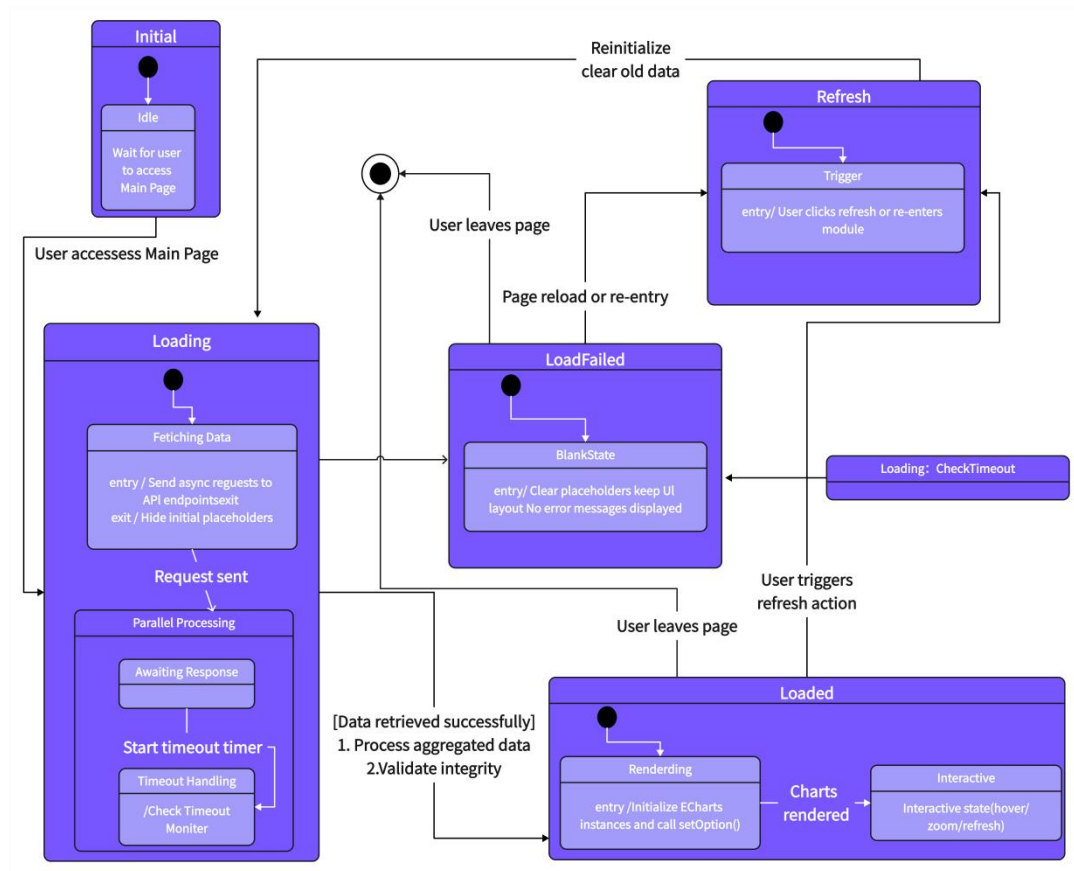
/**
 * Check the total number of bookings with high volume.5a meeting room
 * @return
 */
@GetMapping(#{@"/getLastMeeting"})
public ResultVo getLastMeeting(){
  List<EchartVo> list = meetingBookService.getLatestBook();
  return ResultUtils.success( msg: "Query successful",list);
}

/**
 * Meeting room type
 * @return
 */
@GetMapping(#{@"/getCategoryBook"})
public ResultVo getCategoryBook(){
  List<EchartVo> list = meetingCategoryService.getCategoryBook();
  return ResultUtils.success( msg: "Query successful",list);
}
}
```

Appendix 15: Sequence diagram of PBI-014



Appendix 16: State Machine diagram of PBI-014



Appendix 17: PBI-014 Meeting Room Usage Statistics and Visualization Database Design

```
/**
 * Serial number
 */
@TableId(value = "id", type = IdType.AUTO)
@ExcelProperty("Serial Number")
@ExcelIgnore
private Long id;

@ExcelProperty("Meeting Name")
@ColumnWidth(20)
private String meetingName;

/**
 * Meeting roomid
 */
@TableField("room_id")
@ExcelIgnore
private Integer roomId;

@ExcelProperty("Meeting Room Name")
@ColumnWidth(20)
@TableField(exist = false)
private String name;

/**
 * Using Department
 */
@TableField("dept_id")
@ExcelIgnore
private Integer deptId;

@ExcelProperty("Using Department")
@TableField(exist = false)
@ColumnWidth(20)
private String deptName;
```

```

@Data
@TableName("sys_user")
public class SysUser {
    @TableId(type = IdType.AUTO)
    private Long userId;
    private Long deptId;
    private String username;
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private String password;
    @TableField(updateStrategy = FieldStrategy.IGNORED)
    private String phone;
    @TableField(updateStrategy = FieldStrategy.IGNORED)
    private String email;
    private String sex;
    private String isAdmin;
    //Not part of the user table, Need to exclude
    @TableField(exist = false)
    private String roleId;
    //Is the account expired?(1 Not expired, 0Expired)
    private boolean isAccountNonExpired = true;
    //Is the account locked?(1 Unlocked, 0Locked)
    private boolean isAccountNonLocked = true;
    //Is the password expired?(1 Not expired, 0Expired)
    private boolean isCredentialsNonExpired = true;
    //Is the account available?(1 Available, 0 Delete user)
    private boolean isEnabled = true;
    private String nickName;
    private String avatar;
    //Creation Time
    @JsonFormat(pattern = "yyyy-MM-dd")
    private Date createTime;
    //Update time
    @JsonFormat(pattern = "yyyy-MM-dd")
    private Date updateTime;

    @ExcelProperty("Department name")
    @TableField(exist = false)

```

```

@Data 38 个用法
@Builder(toBuilder = true)
@AllArgsConstructor
@NoArgsConstructor
@Table(name="sys_dept")
public class Dept implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * Serial number
     */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    /**
     * Department Name
     */
    @TableField("dept_name")
    private String deptName;

    /**
     * Department Number
     */
    @TableField("order_num")
    private Integer orderNum;

}

```

```

public class Repair implements Serializable {

    /**
     * Serial Number
     */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    private Long roomId;

    @TableField(exist = false)
    private String name;

    @TableField(exist = false)
    private String nickName;
    /**
     * Repair Details
     */
    @TableField("content_text")
    private String contentText;

    /**
     * Reporter
     */
    @TableField("user_id")
    private Long userId;

    /**
     * Repair Request Time
     */
    @TableField(value = "create_time")
    private Date createTime;

    /**
     * Repair Report Image
     */
    @TableField("image")

```

Appendix 18: The Navicat view of the meeting_book table

id # bigint	room_id # int	dept_id # int	user_id # int	start_time datetime	end_time datetime	status varchar	create_time datetime	use_status varchar(64)	use_desc varchar(255)	apply_desc varchar(255)	process_user varchar(255)	meeting_name varchar(255)
17	1	4	14	2025-03-18 23:00:00	2025-03-19 00:00:00	3	2025-03-27 17:46:33	4	Group project discussion	Approved	Administrator	Startup Brainstorming
19	5	4	14	2025-03-27 22:00:00	2025-03-28 00:00:00	1	2025-03-27 17:50:46	1	Startup competition preparation	Approved	Conference Coordinator 1	Business Pitch Rehearsal
20	2	4	10	2025-03-06 04:00:00	2025-03-06 06:00:00	1	2025-03-27 19:59:11	1	Group 17 Weekly Discussion	Approved	Conference Coordinator 1	Group Assignment Discussion
21	3	4	10	2025-03-13 04:00:00	2025-03-13 06:00:00	1	2025-03-27 20:01:35	1	Group 17 Weekly Discussion	Approved	Conference Coordinator 1	Group Assignment Discussion
22	4	1	18	2025-03-26 22:00:00	2025-03-27 00:00:00	1	2025-03-27 21:01:16	1	Extracurricular math learning	Approved	Conference Coordinator 2	Math Club Workshop 1
23	4	6	18	2025-04-02 22:00:00	2025-04-03 00:00:00	1	2025-03-27 21:02:40	1	Extracurricular math learning	Approved	Conference Coordinator 2	Math Club Workshop 2
24	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 21:12:29	0	Self-directed learning	(Null)	(Null)	Self-directed learning
25	10	9	16	2025-04-22 03:00:00	2025-04-22 01:00:00	0	2025-04-21 21:13:46	0	Self-directed learning	(Null)	(Null)	Self-directed learning
26	8	9	21	2025-03-13 18:00:00	2025-03-13 19:00:00	2	2025-03-28 00:52:18	2	Student union activity	Rejected	Conference Coordinator 1	Club Meeting
29	5	2	12	2025-04-01 08:00:00	2025-04-02 08:00:00	1	2025-04-17 23:53:00	1	For discussion	agree	Conference Coordinator 1	CP202 Discussion
30	6	9	16	2025-04-13 03:00:00	2025-04-13 06:00:00	0	2025-04-14 21:12:29	0	Self-directed learning			Self-directed learning
31	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
32	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
33	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
34	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
35	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
36	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
37	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
38	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
39	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
40	6	9	16	2025-04-22 03:00:00	2025-04-22 06:00:00	0	2025-04-21 03:00:00	0	Self-directed learning			Self-directed learning
41	1	4	13	2025-04-18 23:00:00	2025-04-19 00:00:00	0	2025-04-19 02:16:13	0	Group Discussion	(Null)	(Null)	Group Discussion
43	1	4	13	2025-04-21 22:00:00	2025-04-21 23:00:00	1	2025-04-20 20:31:54	1	Test1	Agree	Administrator	Test1
44	1	4	11	2025-04-23 22:00:00	2025-04-24 06:00:00	1	2025-04-20 21:15:01	0	Test2	(Null)	Conference Coordinator 1	Test2
46	1	4	13	2025-04-22 22:00:00	2025-04-23 01:00:00	1	2025-04-21 17:37:02	1	test	ok	Administrator	test
47	1	4	44	2025-04-24 22:00:00	2025-04-25 01:00:00	1	2025-04-23 23:01:26	1	1111	1111	Administrator	111

Appendix 19: User Interface Design of PBI-014 Meeting Room Usage Statistics and Visualization

```
// Load ECharts statistical charts
const loadCharts = async () => {
  let res = await http.get("/api/home/getEcharts");
  if (res && res.code == 200) {
    const chart1 = global.$echarts.init(myChart.value);
    chart1.setOption({
      tooltip: {
        trigger: 'axis',
        confine: true,
        position: 'top',
        backgroundColor: '#fff',
        borderColor: '#ccc',
        borderWidth: 1,
        textStyle: {
          color: '#333',
        }
      },
      grid: {
        left: "3%",
        right: "4%",
        bottom: "15%",
        containLabel: true,
      },
    });

    res = await http.get("/api/home/getLastMeeting");
    if (res && res.code == 200 && Array.isArray(res.data)) {
      const chart2 = global.$echarts.init(myChart1.value);

      // Validate data structure and extract name and value fields
      const validData = res.data.filter(item => item && item.name && item.value !== undefined);

      const roomNames = validData.map(item => item.name);
      const roomCounts = validData.map(item => item.value);

      chart2.setOption({
        color: ['#BA68C8'],
        tooltip: {
          trigger: "axis",
          axisPointer: { type: "shadow" },
          confine: true,
          position: 'top',
          backgroundColor: '#fff',
          borderColor: '#ccc',
          borderWidth: 1,
          textStyle: {
            color: '#333',
            fontSize: 13,
          }
        },
      });

      res = await http.get("/api/home/getCategoryBook");
      if (res && res.code == 200) {
        const chart3 = global.$echarts.init(myChart2.value);
        chart3.setOption({
          color: purplePalette,
          tooltip: {
            trigger: "item",
            confine: true,
            position: 'top',
            backgroundColor: '#fff',
            borderColor: '#ccc',
            borderWidth: 1,
            textStyle: {
              color: '#333',
              fontSize: 13,
            }
          },
          series: [{
            type: "pie",
            radius: ["40%", "70%"],
            data: res.data,
            label: {
              formatter: '{b|{b}}\n{per|{d}%}',
              rich: {
                b: { fontWeight: 500, color: '#333' },
                per: { color: '#666' }
              }
            }
          }]
        });
      }
    }
  }
}
```

```

<template>
  <el-main :style="{ height: mianHeight + 'px' }">
    <!-- Statistics (First Section) -->
    <el-row :gutter="20" type="flex" justify="center" style="margin-bottom: 40px">
      <el-col :span="6">
        <div class="show-header" style="background: #D88FD8">
          <div class="show-num">{{ total.userTotal }}</div>
          <div class="bottom-text">Total Members</div>
        </div>
      </el-col>
      <el-col :span="6">
        <div class="show-header" style="background: #BA55D3">
          <div class="show-num">{{ total.deptTotal }}</div>
          <div class="bottom-text">Total Departments</div>
        </div>
      </el-col>
      <el-col :span="6">
        <div class="show-header" style="background: #9932CC">
          <div class="show-num">{{ total.roomTotal }}</div>
          <div class="bottom-text">Total Meeting Rooms</div>
        </div>
      </el-col>
      <el-col :span="6">
        <div class="show-header" style="background: #6A0DAD">
          <div class="show-num">{{ total.repairTotal }}</div>
          <div class="bottom-text">Total Repairs Needed</div>
        </div>
      </el-col>
    </el-row>
  </el-main>
</template>

```

```

<template>
  <el-main :style="{ height: mianHeight + 'px' }">
    <!-- Statistical Charts -->
    <div style="display: flex; margin-top: 30px;">
      <el-card style="flex: 1;">
        <template #header>
          <div class="card-header">
            <span style="color: #000000; font-weight: 600;">Department Members</span>
          </div>
        </template>
        <div ref="myChart" style="width: 100%; height: 300px;"></div>
      </el-card>

      <el-card style="margin-left: 20px; flex: 1;">
        <template #header>
          <div class="card-header">
            <span style="color: #000000; font-weight: 600;">Popular Meeting Rooms</span>
          </div>
        </template>
        <div ref="myChart1" style="width: 100%; height: 300px;"></div>
      </el-card>

      <el-card style="margin-left: 20px; flex: 1;">
        <template #header>
          <div class="card-header">
            <span style="color: #000000; font-weight: 600;">Meeting Room Types</span>
          </div>
        </template>
        <div ref="myChart2" style="width: 100%; height: 300px;"></div>
      </el-card>
    </div>
  </el-main>
</template>

```

```

<script setup lang="ts">
import { ref, nextTick, onMounted, reactive } from "vue";
import useInstance from "@/hooks/useInstance";
import http from "@/http/index";

```

```

const mianHeight = ref(0);
const { global } = useInstance();
const myChart = ref<HTMLElement>();
const myChart1 = ref<HTMLElement>();
const myChart2 = ref<HTMLElement>();

```

```

// Statistical data
const getTotal = async () => {
  let res = await http.get("/api/home/getTotal");
  if (res && res.code == 200) {
    Object.assign(total, res.data);
  }
};

```